

OOP

programare orientată pe obiecte

CURS12: INTRO TESTAREA CODULUI

Alexandru Olteanu

{alexandru.olteanu@upb.ro}

Computer Science & Engineering Department

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Universitatea Politehnica București

De ce sa testam?

- gasirea erorilor inainte sa ajunga in productie, la client
- in agile, unde se fac multe deploys, componenta a ciclului de dezvoltare
- testarea automata este pe acelasi nivel cu programarea

Tipuri de teste:

- unit tests: test basic component of code (e.g. method)
- integration tests: test modules work well together (e.g. interaction with db)
- functional tests: get specific value with business significance
- end-to-end tests: check user flow works (e.g. user clicks button, something happens on screen)
- performance tests: check your system under heavy workload (e.g. 10x, 100x, 1000x requests)

Unit testing:

- package structure: 1-on-1 with code structure
- cut dependencies (mocks, in-memory database etc)
- assertions

EduRenewableEnergy on GitHub, the tests branch

what to test and what not to test

Test coverage: cat de bine este acoperit codul de teste?

- linii
- metode
- instructiuni
- ramuri din control flow

Test-Driven Development

- test-first, apoi codul care sa treaca testul
- debugging vs comparatia cu codul care trecea testul in termeni de productivitate
- cuplare mai mica intre obiecte (Madeyski et al)
- mai multe unit tests, dar tot e posibil sa fie nevoie de mai multe functional tests